SIMPLE STUDENT PROJECT GUIDE & VIVA PREPARATION

Campus Event & Crowdfunding Platform

A Complete, Plain-English Walkthrough: How Everything Works Behind the Scenes, What Happens Where, and How to Present it with Confidence.

Project Name:
Campus Event & Crowdfunding Platform

Core Technologies:
HTML5, Tailwind CSS, JavaScript (ES6+), Supabase, Razorpay, Node.js

Purpose of This Guide:
Easy understanding for all team members, step-by-step user journey, and simple viva answers.

Key Features:
Zero Double-Booking, Instant Auto-Refunds, QR Check-in, Verified Certificates & Crowdfunding

What is Inside This Guide

- **Part 1: The 4 Big Building Blocks:** The basic architecture explained with zero buzzwords.
- **Part 2: What, How, Where & When:** The step-by-step journey of a student using the website.
- **Part 3: The 30-Second File Cheatsheet:** Every file explained in exactly 1 plain sentence.
- **Part 4: Ready-to-Speak 2-Minute Pitch:** A simple, high-scoring script for your presentation.
- **Part 5: Top 6 Viva Questions & Answers:** How to answer your teachers naturally without getting nervous.

Part 1: The 4 Big Building Blocks

How It Fits Together

Before diving into code files, here is the bird's-eye view of the 4 tools powering the platform:

🌐 1. The Website (Frontend)

The buttons, cards, colors, and forms that the user sees and clicks on their phone or laptop. Built with clean **HTML**, **Tailwind CSS**, and plain **JavaScript**.

🗄️ 2. Supabase (The Cloud Database)

Think of Supabase as an online, ultra-secure Excel spreadsheet in the cloud where user accounts, event details, ticket numbers, and gatekeeper volunteer lists are stored.

💳 3. Razorpay (The Bank Cashier)

The official payment popup that opens up to collect money securely through UPI (GPay, PhonePe, Paytm), cards, or NetBanking.

🔐 4. Serverless Helpers (api/ folder)

A small private room on the server that keeps our private Razorpay Secret Key safe. It verifies that payment receipts are genuine and triggers instant refunds if seats get full.

> 💡 **The Simple Big Picture**
>
> The student clicks a button on the **Website** $\rightarrow$ **Razorpay** collects the money $\rightarrow$ our **Serverless Helper** verifies the receipt $\rightarrow$ **Supabase** saves the ticket in their account.

Part 2: What, How, Where & When (Step-by-Step)

The Complete User Journey

Step 1: Student Opens the Website & Logs In

`app.js`

WHEN: The moment someone opens the website in their phone or laptop browser.

WHAT: The website checks whether the user is already logged in or if they are a new visitor.

WHERE: `app.js` (runs automatically on every single page).

HOW: `app.js` listens for Supabase login events. If logged in, it pulls their Name, Roll Number, and College, displays their email in the top navigation bar, and sets up Dark Mode if their phone prefers it.

Step 2: Browsing & Instant Search / Filter

`index.html (filterEvents)`

WHEN: The student is on the Home page looking for workshops, sports, or fests.

WHAT: Typing in the search box or clicking category buttons updates the screen instantly with 0ms delay.

WHERE: `index.html` inside the `filterEvents()` JavaScript function.

HOW: All events are downloaded from the database once when the page loads. When you type or filter, your browser simply hides cards that don't match. Because your own device does this locally, it feels instantaneous.

Step 3: Booking a Ticket (Money & Seats)

`index.html + api/create-order.js`

WHEN: The student clicks "Book Ticket" on a paid or free event.

WHAT: Collects payment securely and subtracts 1 from the available seats.

WHERE: `index.html` triggers `api/create-order.js` $\rightarrow$ Razorpay popup $\rightarrow$ `api/verify-payment.js`.

HOW:
1. Our site asks the server: "*Create an official bill for ₹100.*"
2. Razorpay opens on screen and takes UPI payment.
3. Razorpay gives the student a digital receipt.
4. The server mathematically verifies the receipt so nobody can fake it.
5. Supabase saves the new ticket and reduces available seats by 1.

Step 4: The Double-Booking & Instant Auto-Refund Magic

`book_ticket_safe + api/refund-payment.js`

WHEN: **Only 1 ticket is left**, and **two students click book at the exact same millisecond**.

WHAT: Student A gets the ticket. Student B gets their money refunded automatically in seconds.

WHERE: Database function (`book_ticket_safe`) and `api/refund-payment.js`.

HOW:
1. Both students pay on Razorpay.
2. The database acts like a locked turnstile: it lets Student A in first, gives them the ticket, and sets seats to 0.
3. When Student B arrives a fraction of a second later, the database says: "*SOLD OUT!*"
4. Our website immediately catches this error and calls Razorpay's Refund API.
5. Razorpay reverses the money back to Student B's UPI/bank account, and a friendly modal pops up explaining the instant refund.

Step 5: Viewing the Ticket in the Wallet

`wallet.html`

WHEN: The student clicks "My Wallet" to view their pass.

WHAT: Shows a movie-style perforated ticket stub with a secure QR code and calendar link.

WHERE: `wallet.html`

HOW:
• **Perforated Cutouts:** Clean CSS borders that make it look like a physical tearable paper stub.
• **The QR Code:** Does *not* store personal details. It only stores a random 36-character unique ID. Nobody can Photoshop or fake it.
• **Save as PNG:** Clicking "Download Pass" uses `html2canvas` to take a crisp photo of the ticket so students can save it even without internet.

Step 6: Gate Check-in (Volunteer Camera Scanning) `scanner.html`

WHEN: On event day, at the auditorium or gate entry.

WHAT: A student volunteer opens the camera scanner on their smartphone to verify entry.

WHERE: `scanner.html`

HOW:
1. Uses the phone's back camera directly in the browser (no mobile app installation needed).
2. When a QR is scanned, the scanner pauses for 2 seconds so it doesn't scan repeatedly.
3. If someone shows a forwarded screenshot that was already used: Screen flashes **RED** $\rightarrow$ *"ALREADY USED!"*
4. If valid: Screen flashes **GREEN** with a beep sound $\rightarrow$ Shows Name & Roll No, and marks them present in the database.

Step 7: Participation Certificate (The Lock & Key)

`history.html + certificate.js`

WHEN: After the event ends, students visit their History to claim certificates.

WHAT: Only students who were **physically scanned at the door** can download their certificate.

WHERE: Check is in `history.html`; drawing is in `certificate.js`.

HOW:
• If the student skipped the event: Button is locked $\rightarrow$ "🔒 *Certificate Locked - No Attendance Record*".
• If verified present: Clicking download runs `certificate.js`, which acts like an instant digital artist inside the browser. It draws high-res borders, gold seals, and Dean signatures, and downloads a clean certificate.

Step 8: Crowdfunding (Seniors Festival Donations)

`index.html`

WHEN: When clubs or seniors need money for Navratri, farewell, or robot kits.

WHAT: Displays a campaign goal, how much has been collected, and lets anyone donate any custom amount.

WHERE: `index.html` (Donations category & modal).

HOW: Unlike tickets where the price is fixed (e.g. ₹100), the donation modal lets the user type any amount (₹50, ₹100, ₹500). The progress bar automatically fills up in real time as money comes in.

Step 9: Admin Dashboard & Volunteers

`admin.html`

WHEN: When organizers or faculty manage events.

WHAT: Create events, assign junior volunteers to scan tickets, see charts, and export attendance to Excel.

WHERE: `admin.html`

HOW:
• **Volunteer Scanner Access:** Organizers can type a student's email to make them a "Gatekeeper". That student can scan tickets *only* for that single event (no admin power).
• **Charts:** Uses Chart.js to show attendee breakdown by gender, branch, and college year.
• **Excel Export:** Downloads a clean CSV list of attendees with IST timestamps.

Part 3: The 30-Second File Cheatsheet

Quick Revision

If an examiner points to any file on screen, here is your 1-line answer:

File Name	What It Does in Simple Words
`app.js`	Handles user login/logout, dark mode, and mobile drawer menu opening.
`index.html`	The main home page: search events, book tickets, and donate to campaigns.
`wallet.html`	Shows booked tickets with secure QR codes and Google Calendar buttons.
`history.html`	Shows past events attended or missed, with the locked/unlocked certificate.
`admin.html`	Organizer dashboard to create events, see charts, add scanner volunteers, and export CSVs.
`scanner.html`	Uses the phone's back camera to scan tickets at the door and prevent duplicates.
`certificate.js`	Draws official participation certificates directly inside the browser using HTML5 Canvas.
`api/create-order.js`	Tells Razorpay how much money to ask the student for.
`api/verify-payment.js`	Mathematically checks that the payment receipt is genuine and not faked.
`api/refund-payment.js`	Automatically sends money back if someone pays for a sold-out seat.

Part 4: Ready-to-Speak 2-Minute Pitch

For Your Presentation

🎙️ Your Presentation Speech (Memorize or Adapt This):

*"Good morning, respected professors. Today we are presenting our project: the **Campus Event & Crowdfunding Platform**."*

1. The Problem:
"In our college, event registrations are handled through Google Forms, money is sent over WhatsApp screenshots, and seniors struggle to collect cash donations for fests like Navratri. There is no verification, leading to proxy attendance and manual confusion."

2. Our Solution:
"We built an all-in-one web portal with 5 main pillars: First, students can browse and register for technical and cultural events in one place. Second, clubs and seniors can launch crowdfunding campaigns with transparent live progress bars. Third, every registered student receives a digital ticket in their wallet with a unique QR code. Fourth, event volunteers can turn their smartphones into gatekeeper scanners to verify tickets at the auditorium door. And fifth, certificates are automatically locked and only unlock for students who physically attended."

3. Special Technical Highlights:
"We integrated real Razorpay online payments. We also solved the double-booking problem: if two people pay for the last ticket, our database prevents double-booking and our system automatically triggers an instant refund. The entire website is fully mobile-optimized with Dark Mode support. Let us now show you a quick live demo."

Part 5: Top 6 Viva Questions & Answers

Examiner Cheat-Sheet

Q1: Can someone take a screenshot of their ticket and send it to 5 friends for free entry?

Answer: *"No, Sir. The QR code only contains a unique ticket ID. The moment the first student is scanned at the door, the database marks it as 'Already Used'. If a friend tries to scan the same screenshot 2 minutes later, the scanner screen flashes RED with an error sound showing that this ticket was already used."*

Q2: How did you prevent double-booking when only 1 ticket is left?

Answer: *"We handled this in our Supabase database using a stored procedure called `book_ticket_safe`. When someone is booking, the database temporarily locks that seat for a fraction of a second to finish the booking before letting the next person in. If someone's payment goes through but the seat was already taken, our system automatically calls Razorpay's API to issue an instant refund."*

Q3: Why did you build the QR scanner in the browser instead of making an Android APK?

Answer: *"Because college volunteers change for every event. Asking 20 student volunteers to download an APK file takes time and setup. With our web scanner, they just open the website on their phone's Chrome or Safari browser, and it activates their camera immediately with zero installation."*

Q4: How do you stop students from editing JavaScript in DevTools to fake a payment?

Answer: *"The frontend website never decides if a payment is real. When Razorpay takes money, it generates a cryptographic receipt signature. Our backend helper file (`api/verify-payment.js`) checks that signature against our secret key on the server. If it doesn't match, the ticket is rejected."*

Q5: What makes the Crowdfunding feature different from normal event tickets?

Answer: *"Event tickets have a fixed price and limited seats. Crowdfunding campaigns have no seat limits and accept any custom donation amount (e.g. ₹50, ₹100, or ₹500) with a live progress bar showing how close the seniors are to their festival goal."*

Q6: Why generate the certificate in the browser instead of on the server?

Answer: *"Server-side PDF generators take a lot of server memory and slow down when 500 students download at the same time. Our browser Canvas engine draws the certificate locally on the student's phone in under 80 milliseconds with zero server load and zero cloud costs."*